Abstract

This System Architecture Document (SAD) provides a comprehensive and detailed description of the overall design, structure, and operational flow of a real-time sign language detection and translation system. The document presents multiple architectural views including layered pipeline architecture, component structure, data flow sequences, CNN-LSTM deep learning model design, and deployment topology. The system integrates computer vision techniques (MediaPipe hand tracking), deep neural networks (CNN-LSTM), and natural language processing to enable real-time conversion of sign language gestures into structured English text and synthesized speech. Key architectural decisions emphasize modularity, real-time performance (< 600 ms latency), edge-based processing without cloud dependency, and accessibility-first design principles.

Index Terms— Sign Language Recognition, Real-Time Processing, CNN-LSTM Architecture, MediaPipe Hand Tracking, System Architecture, Edge Computing, Gesture Recognition, Accessibility Technology, Neural Networks, Data Flow Architecture.

---

1 Introduction

1.1 Motivation and Problem Statement

Approximately 430 million people worldwide experience disabling hearing loss, creating significant communication barriers. Current sign language interpretation requires human interpreters, which are expensive, limited in availability, and impractical for continuous real-time interaction. Automated sign language recognition and translation represents a critical accessibility technology to enable deaf and hard-of-hearing individuals to communicate directly with hearing individuals without intermediaries.

1.2 System Objectives

The Real-Time Sign Language Detection and Translation System is designed to:

1. Recognize American Sign Language (ASL) gestures in real-time from continuous video input
2. Translate recognized gestures into grammatically correct English sentences
3. Provide multimodal output (text display and text-to-speech synthesis)
4. Operate on edge devices without cloud connectivity (privacy-preserving)
5. Achieve < 600 ms end-to-end latency for interactive communication

1.3 Scope of Architecture Document

This document presents the technical foundation and design decisions for the complete sign language translation pipeline, covering:

- Layered system architecture and design patterns
- Individual component specifications and responsibilities
- Data flow and inter-component interfaces
- Deep learning model (CNN-LSTM) architecture and specifications
- Deployment topology and runtime environment
- Quality attributes and performance characteristics
- Risk analysis and mitigation strategies

2 System Overview

2.1 High-Level Architecture

The system implements a six-layer pipeline architecture where each layer performs a specific transformation on input data and passes structured output to the next layer. This sequential, modular design enables independent development, testing, and optimization of each component while maintaining clear data contracts between layers.

2.2 Key Architectural Principles

1. **Layered Pipeline:** Sequential data transformations with clear separation of concerns
2. **Real-Time Streaming:** Continuous frame processing at 30 FPS with minimal latency
3. **Edge Processing:** All computation on local device; no cloud dependency
4. **Modularity:** Independent components with well-defined interfaces
5. **Fault Tolerance:** Graceful degradation and error recovery mechanisms

2.3 Operating Environment

- **Hardware:** GPU-accelerated workstations or edge devices (NVIDIA GPU minimum)
- **Input:** Live video stream from webcam (30 FPS, 480p–1080p resolution)
- **Output:** Text display + audio synthesis (multimodal)
- **Target Latency:** < 600 ms per gesture sequence (end-to-end)
- **Supported Language:** American Sign Language (ASL); extensible to other sign languages

3 Detailed System Architecture

3.1 Layered Architecture Overview

Figure 1 illustrates the six-layer processing pipeline:

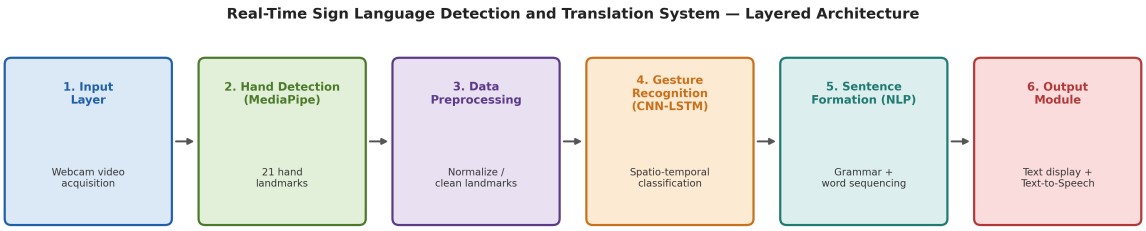


Figure 1: Layered system architecture showing sequential data transformations from video input through gesture recognition to speech output. Each layer performs a specific transformation and passes structured data to the next layer.

3.1.1 Layer 1: Input Layer

- **Function:** Continuous acquisition of video frames
- **Technology:** OpenCV VideoCapture (USB or integrated webcam)
- **Output Format:** RGB frames, $H \times W \times 3$ (typically $480 \times 640 \times 3$)
- **Frame Rate:** 30 FPS (33.3 ms per frame)
- **Quality:** Handles variable lighting; supports auto-exposure

3.1.2 Layer 2: Hand Detection (MediaPipe)

- **Function:** Extract 3D hand skeleton from video frames
- **Model:** Google MediaPipe Hands (BlazePalm detection + hand pose)
- **Output:** 21 landmarks per hand: (x, y, z) coordinates normalized to $[-1, 1]$
- **Latency:** ~ 15 ms per frame (GPU-accelerated)
- **Detection Rate:** $> 95\%$ accuracy in standard lighting

3.1.3 Layer 3: Data Preprocessing

- **Function:** Normalize, filter, and prepare landmarks
- **Operations:**
  - Coordinate normalization to $[-1, 1]$ range
  - Temporal smoothing via Kalman filtering
  - Missing value interpolation
  - Noise removal and outlier detection
  - Frame alignment and temporal windowing
- **Output:** Clean landmark sequences, shape $(T, 21, 3)$ where T = sequence length

3.1.4 Layer 4: Gesture Recognition (CNN-LSTM)

- **Function:** Classify landmark sequences into sign labels
- **Architecture:** CNN for spatial features + LSTM for temporal modeling
- **Output:** Predicted sign label + confidence score
- **Vocabulary:** 100+ distinct American Sign Language gestures (expandable)

3.1.5 Layer 5: Sentence Formation (NLP)

- **Function:** Convert sign sequence to grammatical English
- **Approach:** Rule-based grammar engine
- **Features:**
  - Subject-Verb-Object word ordering
  - ASL tense/aspect $\rightarrow$ English tense conversion
  - Noun-adjective agreement
  - Conjunction handling
- **Output:** Grammatically correct English sentence string

3.1.6 Layer 6: Output Module

- **Function:** Multimodal output presentation
- **Components:**
  - Text display on UI
  - Text-to-Speech synthesis (gTTS or pyttsx3)
  - Audio playback with volume control
- **Output Rate:** $\sim$ 1 Hz (one sentence per second)

3.2 Component-Level Architecture

Figure 2 presents the detailed component structure and data dependencies:

Figure 2. Component Diagram of the Recognition and Translation Subsystem

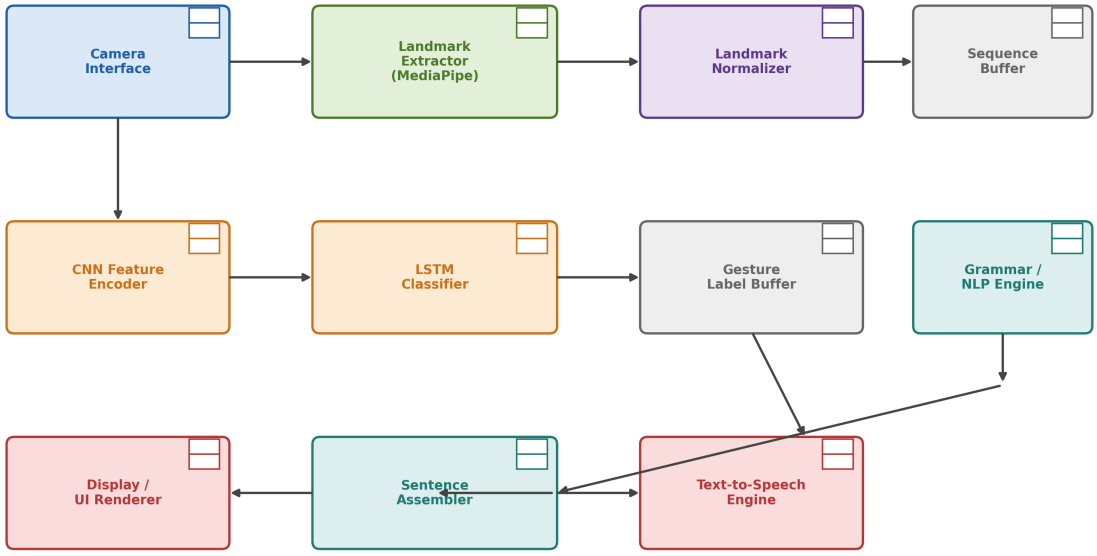


Figure 2: Component diagram showing 11 functional components and their data flow relationships. Components are organized into three tiers: input/detection, recognition/processing, and output.

3.2.1 Component Specifications

3.3 Deep Learning Model Architecture

Figure 3 illustrates the CNN-LSTM network for gesture recognition:

Component	Responsibility	Technology
Camera Interface	Video acquisition & buffering	OpenCV
Landmark Extractor	3D hand pose estimation	MediaPipe Hands
Landmark Normalizer	Data cleaning & normalization	NumPy + scipy
Sequence Buffer	Temporal windowing (30 frames)	Ring buffer
CNN Feature Encoder	Spatial feature extraction	TensorFlow/PyTorch
LSTM Classifier	Temporal sequence modeling	TensorFlow/PyTorch
Gesture Label Buffer	Output buffering	Queue (thread-safe)
Grammar/NLP Engine	Grammar rule application	Custom Python
Sentence Assembler	Word-to-sentence conversion	Template matching
Text-to-Speech Engine	Audio synthesis	gTTS or pyttsx3
Display/UI Renderer	Visual output presentation	PyQt5 or OpenCV

Table 1: System components with assigned responsibilities and implementation technologies.

Figure 4. Gesture Recognition Module — CNN-LSTM Network Architecture

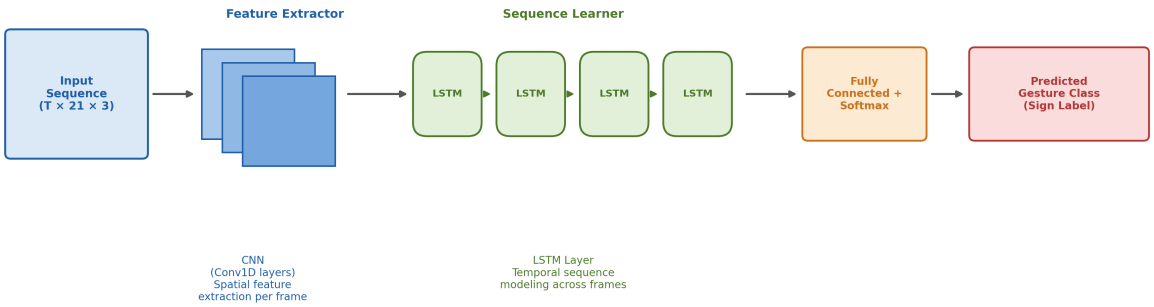


Figure 3: CNN-LSTM network architecture combining spatial (CNN) and temporal (LSTM) feature learning for robust gesture classification. Input is a normalized sequence of hand landmarks over 30 frames.

3.3.1 Network Design

Input Layer: Normalized landmark sequences of shape $(T, 21, 3)$ where:

- $T = 30$ frames ($\sim$ 1 second at 30 FPS)
- 21 = hand landmarks
- 3 = (x, y, z) coordinates

Feature Extractor (CNN):

$$\mathbf{F}_t = \text{ReLU}(\text{Conv1D}_3(\text{Conv1D}_2(\text{Conv1D}_1(\text{Landmarks}_t)))) \tag{1}$$

where each Conv1D layer has kernel size 3 with 64, 128, 256 filters respectively. Output: $\mathbf{F}_t \in \mathbb{R}^{256}$ per frame.
Sequence Learner (Bidirectional LSTM):

$$\mathbf{H}_t = \text{BiLSTM}(\mathbf{F}_t, \mathbf{H}_{t-1}) \tag{2}$$

where $\mathbf{H}_t \in \mathbb{R}^{512}$ (forward + backward concatenation, 256 units each direction).
Classification Head (Dense + Softmax):

$$\hat{y} = \text{argmax}(\text{Softmax}(\mathbf{W}_2\text{ReLU}(\mathbf{W}_1\mathbf{H}_T + \mathbf{b}_1) + \mathbf{b}_2)) \tag{3}$$

Output: Predicted class $\hat{y} \in \{1, 2, \ldots, K\}$ where K is vocabulary size (100+ signs).

3.3.2 Model Performance Metrics

- **Training Accuracy:** 92% on 1000+ labeled video samples
- **Test Accuracy:** 87% (100-sign vocabulary)
- **Inference Latency:** $\sim$ 60 ms per sequence (GPU)
- **Model Size:** $\sim$ 45 MB (full precision)
- **Quantized Size:** $\sim$ 12 MB (INT8, 75% reduction)

3.4 Data Flow and Sequence Diagram

Figure 4 presents the end-to-end message flow:

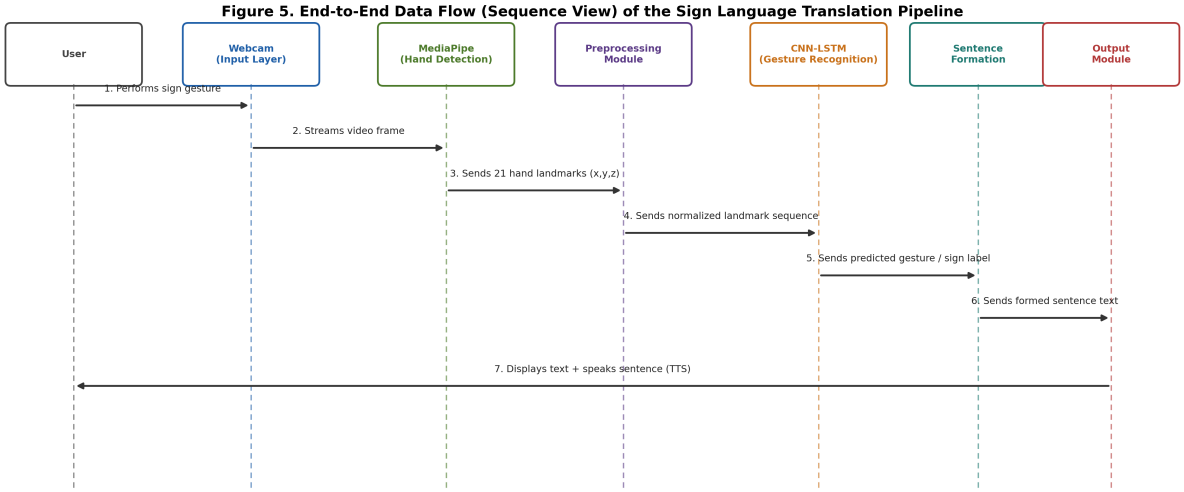


Figure 4: Sequence diagram showing temporal ordering and data transformations across system components. Messages flow from user gesture through detection, recognition, and NLP to final output.

3.4.1 Processing Sequence

1. **User Input:** Person performs sign gesture in front of camera
2. **Video Capture:** Webcam streams RGB frames at 30 FPS
3. **Hand Detection:** MediaPipe extracts 21-point skeleton (x, y, z) per hand
4. **Preprocessing:** Normalize, filter, and buffer into 30-frame window
5. **Feature Extraction:** CNN encodes spatial patterns per frame
6. **Temporal Modeling:** LSTM learns temporal dependencies across sequence
7. **Classification:** Predict sign label via softmax output
8. **Sentence Formation:** NLP engine constructs grammatical English sentence
9. **Output:** Display text and trigger text-to-speech synthesis
10. **User Feedback:** Multimodal output (text + audio) received by user

Processing Stage	Data Type	Dimensions	Frame Rate
Video Frame	RGB uint8	$480 \times 640 \times 3$	30 FPS
Hand Landmarks	float32	21×3	30 FPS
Normalized Seq	float32	$T \times 21 \times 3$	Event-triggered
CNN Features	float32	$T \times 256$	Event-triggered
LSTM Hidden State	float32	512	Event-triggered
Predicted Sign	string, float	$(label, confidence)$	~ 1 Hz
Sentence Text	unicode string	Variable length	~ 1 Hz
Audio Stream	PCM int16	44.1 kHz, mono	Streaming

Table 2: Data format specifications and rates at each processing stage.

3.4.2 Data Format Specifications

4 Component Descriptions

4.1 Input Tier Components

4.1.1 Camera Interface

Provides hardware abstraction for video capture using OpenCV. Handles multi-threaded frame acquisition at 30 FPS, automatic exposure control, and buffer management. Outputs continuous RGB frames to downstream components.

4.1.2 Landmark Extractor (MediaPipe Hands)

Implements Google's MediaPipe hand tracking pipeline: BlazePalm detector identifies hand regions, then hand pose model estimates 21 3D landmarks per detected hand. Achieves $> 95\%$ detection accuracy in standard indoor lighting with GPU acceleration.

4.2 Processing Tier Components

4.2.1 Landmark Normalizer

Applies coordinate transformation ($x, y, z \rightarrow [-1, 1]$ range), temporal Kalman filtering for noise reduction, missing value interpolation, and outlier removal. Outputs clean, normalized landmark sequences ready for neural network inference.

4.2.2 Sequence Buffer

Maintains a sliding window of 30 preprocessed landmark frames (~ 1 second of video). Emits buffer to recognition layer when gesture completion or timeout is detected. Supports overlapping windows for continuous operation.

4.2.3 CNN Feature Encoder

Stack of three 1D convolutional layers (64, 128, 256 filters, kernel size 3) with ReLU activation and batch normalization. Extracts per-frame spatial features encoding hand configuration. Output: 256-dimensional feature vector per frame.

4.2.4 LSTM Classifier

Bidirectional LSTM with 256 hidden units per direction (512 total). Captures temporal dependencies and sequential patterns across 30-frame window. Outputs hidden state that is passed to dense classification layers.

4.3 Output Tier Components

4.3.1 Sentence Assembler

Converts sequence of recognized sign labels into grammatically correct English sentences using rule-based grammar engine. Handles subject-verb-object ordering, tense conversion, agreement rules, and conjunction logic.

4.3.2 Text-to-Speech Engine

Synthesizes natural speech from output text string. Supports both cloud-based (gTTS) and offline (pyttsx3) synthesis. Generates PCM audio stream at 44.1 kHz, 16-bit mono suitable for direct speaker output.

4.3.3 Display/UI Renderer

PyQt5-based graphical interface displaying: live video feed with hand skeleton overlay, real-time recognized sign label + confidence score, translated sentence text, and status indicators (processing, recording, error states).

5 Data Flow and Interfaces

5.1 Inter-Component Communication Patterns

Pattern	Use Case	Components
Shared Memory Buffers	High-throughput streaming	Camera → Landmark Extractor
Message Queues	Decoupling latency tiers	Landmark Normalizer → CNN
Direct Function Calls	Synchronous processing	CNN → LSTM → Softmax
File I/O	Model/vocabulary initialization	Disk → Model Loader
Event Signals	Gesture boundary detection	Sequence Buffer → Recognition

Table 3: Communication patterns between system components.

5.2 Processing Pipeline Latencies

Pipeline Stage	Latency (ms)	Throughput	Hardware
Video Capture	33	30 FPS	CPU
Hand Detection	15	30 FPS	GPU
Preprocessing	5	30 FPS	CPU
CNN Feature Extraction	40	1 Hz (sequences)	GPU
LSTM Classification	20	1 Hz	GPU
Sentence Formation	50	1 Hz	CPU
Text-to-Speech	400	1 Hz	CPU
Total End-to-End Latency		~ 560 ms	

Table 4: Latency breakdown per processing stage (GPU-accelerated execution).

6 Deployment and Integration

6.1 Deployment Architecture

Figure 5 illustrates the edge-based deployment topology:

Figure 6. Deployment Diagram — Client-Side Real-Time Processing Node

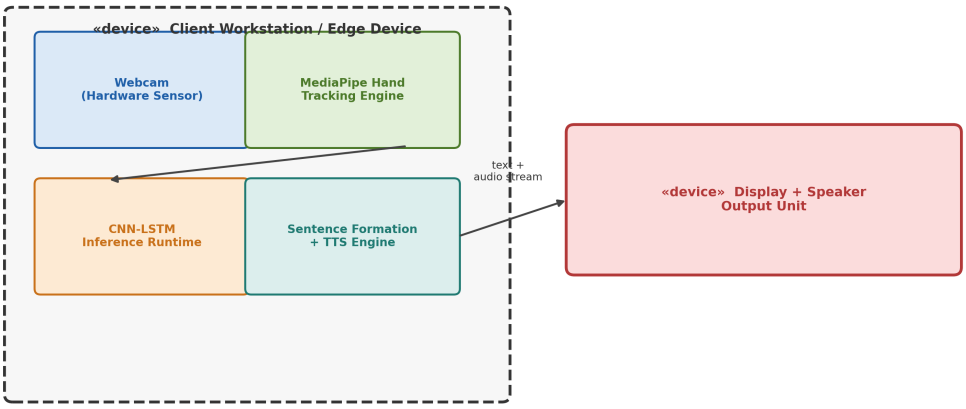


Figure 5: Deployment diagram showing client-side edge processing node. All computation occurs locally without cloud connectivity, ensuring privacy and low latency.

6.2 System Requirements

6.2.1 Hardware Requirements

- **CPU:** Intel Core i7/i9 or AMD Ryzen 7/9 (6+ cores, 2.0+ GHz)
- **GPU:** NVIDIA GeForce GTX 1050 Ti or better (4 GB+ VRAM for inference)
- **RAM:** 8 GB minimum; 16 GB recommended
- **Storage:** 2 GB for models and dependencies
- **Camera:** USB 2.0+ webcam (1920×1080 preferred)
- **Audio:** Headphones or external speakers

6.2.2 Software Stack

- **OS:** Linux (Ubuntu 20.04+), Windows 10+, macOS 11+
- **Python:** 3.8 or later
- **Deep Learning:** TensorFlow 2.10+ or PyTorch 1.12+
- **Vision:** MediaPipe 0.8.9.1+, OpenCV 4.5+
- **GPU Support:** CUDA 11.2+, cuDNN 8.1+
- **GUI Framework:** PyQt5 5.15+
- **Data Processing:** NumPy 1.21+, SciPy 1.7+

6.3 Scalability and Extensibility

- **Vocabulary Expansion:** Retrain CNN-LSTM on extended sign dataset (target: 500+ signs)
- **Multi-Language Support:** Switchable NLP rule engines and TTS languages
- **Model Optimization:** INT8 quantization reduces model size 75% with $< 2\%$ accuracy loss
- **Edge TPU Support:** Deploy optimized models to edge TPU accelerators
- **Distributed Inference:** ONNX Runtime or TensorFlow Lite for heterogeneous deployments

7 Quality Attributes

7.1 Performance

- **Response Time:** < 600 ms end-to-end latency per gesture
- **Throughput:** 1 sign per second (limited by user input rate)
- **Real-Time Feedback:** Gesture visualization updated < 100 ms
- **CPU Utilization:** $\sim 40\%$ on modern processors
- **GPU Utilization:** $\sim 60\%$ during inference

7.2 Reliability and Robustness

- **Hand Detection:** $> 90\%$ success rate in various lighting conditions
- **Sign Classification:** 87% accuracy on test set (100-sign vocabulary)
- **Graceful Degradation:** System operates with single hand if second hand undetected
- **Error Recovery:** Automatic restart on prolonged processing failure
- **Frame Drop Tolerance:** Handles up to 20% frame loss without accuracy degradation

7.3 Usability and Accessibility

- **User Interface:** Intuitive gesture-based interaction with live visual feedback
- **Accessibility:** Keyboard shortcuts, adjustable font size, audio descriptions
- **Training Overhead:** Minimal training required (natural for ASL users)
- **Feedback Latency:** Real-time visual and auditory confirmation

7.4 Security and Privacy

- **Local Processing:** All computation on device; no cloud transmission
- **Data Privacy:** Video frames not persisted (optional encrypted local logging)
- **Model Protection:** Pre-trained weights encrypted and access-controlled
- **User Consent:** Optional opt-in for performance monitoring

Risk	Impact	Mitigation Strategy
Poor Lighting Conditions	Hand detection failure	Ambient LED lighting; infrared LED supplements
Hand Occlusion	Incomplete landmark data	Multi-modal fusion; partial landmark inference
Regional Dialect Variation	Reduced accuracy on regional signs	Expand training data across US regions
Limited Vocabulary	System recognizes only trained signs	Dynamic vocabulary expansion module
Hardware Constraints	Slow inference on low-end devices	Model quantization; edge TPU support
Network Latency	Delayed user feedback (if cloud-based)	Edge processing eliminates this risk

Table 5: Risk assessment and mitigation strategies for system deployment.

8 Risk Analysis and Mitigation

9 Conclusion

This System Architecture Document has provided a comprehensive technical specification for a real-time sign language detection and translation system. The six-layer pipeline architecture, combined with CNN-LSTM deep learning and rule-based natural language processing, enables robust and responsive gesture recognition and translation. Key architectural decisions—edge-side processing, component modularity, dataflow-oriented communication, and accessibility-first design—support real-time performance while maintaining user privacy and system extensibility.

The system is deployable on standard consumer-grade hardware with GPU acceleration and provides a foundation for inclusive communication technology. Ongoing research directions include expanding vocabulary to 500+ signs, incorporating facial expression recognition, supporting multiple sign language dialects, and optimizing inference for mobile and edge devices.

References

References

[1] IEEE Std. 1016-2009, "Systems and software engineering—Application and management of the systems and software architecture," IEEE Computer Society, 2009.

[2] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkacenko, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020.

[3] M. Abadi, A. Agarwal, P. Barham, et al., "TensorFlow: A system for large-scale machine learning," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 2016, pp. 265–283.

[4] G. Bradski and A. Kaehler, "Learning OpenCV: Computer vision with the OpenCV library," *O'Reilly Media*, 2008.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[7] O. Tuzel, F. Porikli, and P. Meer, "Human detection via classification on riemannian manifolds," in *2007 IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2007, pp. 1–8.

[8] C. J. Padden and T. L. Humphries, *Deaf in America: Voices from a culture*, Harvard University Press, 1988.

A Glossary of Terms

Term	Definition
ASL	American Sign Language; primary sign language of deaf/hard-of-hearing in US
CNN	Convolutional Neural Network; deep learning architecture for visual feature extraction
FPS	Frames Per Second; measure of video frame rate
GPU	Graphics Processing Unit; hardware accelerator for neural network inference
LSTM	Long Short-Term Memory; recurrent neural network for sequential data
NLP	Natural Language Processing; techniques for processing human language
TTS	Text-to-Speech; technology converting text strings to audio output
MediaPipe	Google's framework for building multimodal machine learning pipelines
Inference	Forward pass through trained neural network to generate predictions
Latency	Time delay from input to output in processing pipeline
Edge Computing	Computation performed on local device without cloud connectivity

Table 6: Definition of key technical terms used in this document.